Entendendo o Desafio

Agora é a sua hora de praticar, aprender fazendo e montar um pipeline de pagamentos que roda em tempo real. Do banco de dados até o alerta de fraude, tudo no **Confluent Cloud**.

Modelo de Entrega:
Pipeline de pagamentos: entrega de exemplo

O Expert Mateus apresenta o Desafio e grava a execução inteira, do zero. Esse PDF é o esqueleto de uma entrega nota alta: os trechos marcados com `<sua evidência aqui>` são o que você troca pela saída do seu pipeline.

O Que Criar

Um pipeline de pagamentos de ponta a ponta. Os dados nascem em um **Postgres**, entram no Confluent Cloud por **CDC**, passam pelo **Flink SQL**, que junta as informações e marca as transações suspeitas, e seguem para quem vai consumir.

Você não precisa inventar os dados, eles são sintéticos e já vêm prontos. O domínio também pode ser outro, se pagamentos não é o seu tema. O que a avaliação olha é o caminho do dado.

Como Fazer

O caminho tem cinco camadas:

1. **Fundação.** Criar o ambiente e o cluster, com uma identidade para quem escreve e outra para quem lê;
2. **Contrato.** Registrar o formato das mensagens no Schema Registry e testar se ele aguenta uma mudança de campo;
3. **Ingestão.** Ligar o conector de CDC ao Postgres e ver o insert, o update e o delete chegando como evento;
4. **Processamento.** No Flink SQL, enriquecer a transação com os dados da conta e escrever a regra de fraude: três transações no mesmo cartão em 60 segundos viram um alerta;
5. **Operação.** Olhar as métricas, o acesso e o custo do que você subiu.

Se você ainda está começando, tudo bem. Suba uma camada por vez e confira antes de passar para a próxima. Um pipeline menor, rodando e bem explicado, vale mais que uma arquitetura ambiciosa que ninguém consegue subir de novo.

Ideias para Evoluir

- Trocar pagamentos por outro domínio, como IoT ou personalização em tempo real;
- Escrever uma segunda regra de fraude, com outra janela e outro critério;
- Guardar o evento com defeito em uma fila separada, em vez de deixar o conector parar;
- Materializar os tópicos como tabelas **Apache Iceberg** com **Tableflow**;
- Enriquecer o stream com um modelo de IA que pontua o risco de cada transação.

Uma Ajuda durante o Caminho

Se travar, o DIO Agent ajuda a destravar o raciocínio:

```
None
Estou fazendo um Desafio de Projeto de pipeline em tempo real.
Meu pipeline leva dados de um Postgres para o Confluent Cloud
por CDC, processa com Flink SQL e emite um alerta de fraude.

Me ajude a entender o que o conector de CDC precisa do banco
antes de subir, e por onde começar se eu travar na primeira
camada.

Não quero o comando pronto para copiar. Quero entender o que
acontece em cada etapa para montar o meu pipeline.
```

O Que Entregar

Um repositório público no GitHub com o pipeline funcionando: os arquivos de configuração, os scripts SQL e o README. No README, conte:

- O que cada camada faz, com a evidência de que ela rodou: print, saída de comando ou o próprio evento;
- Por que você escolheu cada peça do caminho;
- Como alguém sobe o seu pipeline do zero, e como derruba tudo no fim;
- Quanto custou rodar.

Evidência é o que separa a entrega que se prova da que só se descreve. Rode o comando, copie a saída, cole no README.

Antes de submeter, confira:

- Cada camada tem evidência, e não sobrou nenhum `<sua evidência aqui>`;
- Outra pessoa sobe o seu pipeline seguindo só o que está escrito;
- Nenhuma senha ou chave foi versionada, e o `.env.example` ficou no lugar do `.env`;
- O link enviado é o do repositório, não o de um arquivo dentro dele;
- O repositório está na sua conta e público, com nome em minúsculas e sem acento.

Resultado Esperado

Ao final, você terá um pipeline de streaming completo no portfólio, do banco de dados até o alerta. É o tipo de projeto que sustenta uma conversa técnica inteira: por que CDC e não batch, por que essa janela, quanto custou rodar.

Bons estudos e bom projeto 🚀